
Reframe: A Live Diminished- and Augmented-Reality Room Editor with Deterministic Spatial Authority

Team Reframe

Julian Schmidt

Filip Paunovic

Hayat Öner

Lorenz Benke

Abstract

Reframe is a live diminished- and augmented-reality (DR/AR) room editor. The primary experience runs in a native iPhone application: the real camera feed remains the photoreal background, ARKit keeps the world metrically stable, and the system draws only the virtual content required to make an edit believable, namely multi-surface background reveal geometry, occlusion meshes, replacement furniture, contact shadows, and interface elements. Cloud services continuously improve spatial understanding but never enter the 60 Hz rendering loop. This report is a systems and architecture paper. It documents the complete Reframe design: three product modes, two parallel geometry tracks, a capability-specific readiness model, and a compare-and-swap transaction system in which language models (GPT-5.6 Sol for planning, GPT Realtime 2.1 for voice) are strictly advisory while deterministic code owns identity, geometry, revisions, commit, and restore. It further specifies every external contract (coordinate and intrinsics conventions, the binary FramePacket, the durable `.rfcap` capture format, EditKit artifacts, the canonical scene graph), every subsystem (capture, compositing, fast silhouette geometry, depth providers and metric alignment, TSDF fusion, SAM 3.1 semantics, plane atlases and reveal generation, catalog acquisition and retrieval, gateway, agent, voice, and the Next.js Mode B0 client), and the infrastructure that executes them (Docker topology, SQLite-WAL persistence, GPU lane scheduling, transport and authentication, and privacy and retention rules). We close with an explicit account of implementation state, separating contracts that are implemented and test-verified in the repository from acceptance gates that remain open. All performance numbers in this report are targets to be measured, not measured results.

1 Introduction

Editing a real room in place requires solving two problems that are usually treated separately: diminished reality (convincingly showing what is behind a real object that has been removed) and augmented reality (convincingly inserting virtual objects that occlude and are occluded correctly). Reframe combines both in one product under a conservative systems posture: the camera feed is never modified per pixel, all scene mutations flow through a deterministic transaction authority, and every piece of machine-learning inference sits behind a typed, replaceable provider boundary off the interaction-critical path.

1.1 Product definition

Reframe is a live diminished- and augmented-reality room editor. The primary experience is a native iPhone application. The real camera feed remains the photoreal background, ARKit keeps the world stable, and Reframe draws only the virtual content needed to make an edit believable: background reveal surfaces, occlusion geometry, replacement furniture, shadows, and UI. Cloud services continuously improve spatial understanding but never enter the 60 Hz rendering loop.

The product exposes exactly four user-visible editing operations:

1. **Place:** place one catalog asset on a detected floor at the reticle/tap location.
2. **Replace:** replace the staged freestanding armchair or small side table with a normalized asset that physically fits and visually covers the old object.
3. **Remove:** hide the staged object and reveal a precomputed multi-surface background bundle.
4. **Restore / undo:** restore the real object or undo the most recent committed transaction, locally and immediately.

The primary operation is **replace**. Empty removal is enabled only when its reveal-quality requirements pass (Section 11.4).

1.2 The three product modes

The architecture defines three product modes, summarized in Figure 1:

- **Mode A (Live AR).** Native SwiftUI + ARKit + RealityKit/Metal on iPhone. Place, replace, remove, and restore while looking through the live camera. This is the product’s primary mode.
- **Mode B0 (Scan, process, and continue).** Every capture is recorded in `.rfcap`; the same session can be processed or replayed later and edited from the Next.js application. Ordinary uploaded video is also supported. Mode B0 is a guaranteed core-release product, not a developer utility.
- **Mode B1 (Photoreal twin).** An immutable capture snapshot may be globally consolidated and optimized into a Gaussian-splat scene for the web after the live and replay paths meet their requirements.

Mode B1 never touches the critical path. It is disabled until all live and replay requirements pass, it never consumes live-path compute, and it may only replace a render skin; it cannot rewrite canonical object IDs, support relations, transactions, or undo history. This restriction is a permanent product invariant rather than a scheduling decision: the core product depends on deterministic scene authority, and a photoreal skin with write access to that authority would turn a cosmetic upgrade into a correctness risk.

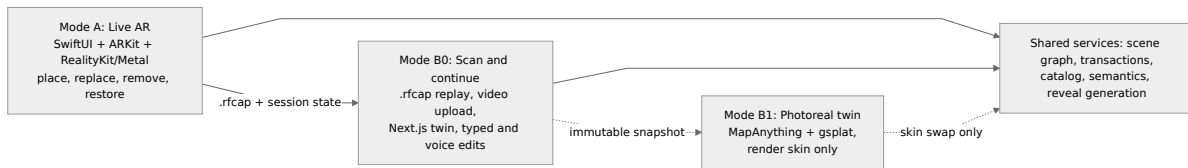


Figure 1: The three product modes and their data handoffs. Mode A hands the durable `.rfcap` capture and session state to Mode B0; Mode B0 may hand an immutable snapshot to Mode B1. All modes share the same deterministic scene, transaction, catalog, semantics, and reveal services. Mode B1 may only swap a render skin and never rewrites canonical state.

1.3 Contributions of this report

This is a systems and architecture paper. Its contributions are: (i) a complete architectural specification of a live DR/AR editor built around deterministic spatial authority, in which language models are advisory and code owns identity, geometry, revisions, commit, and restore; (ii) a two-track geometry design that removes learned dense reconstruction from the interaction-critical path; (iii) precise external contracts for coordinates, intrinsics, binary framing, durable capture, artifacts, and the scene graph, which make every runtime boundary typed and testable; (iv) a capability-specific readiness and quality-gate model for diminished reality that does not promise instant removal; and (v) an implementation-state assessment that separates specified design, repository-verified behavior, and open acceptance gates (Section 16).

Every technical claim in this report is grounded in the Reframe Master Technical Prompt (the product and architecture authority) or in the repository implementation. Where behavior is unspecified, we state that explicitly rather than inventing it.

2 Product boundary and scope

2.1 Initial operating envelope

The core release targets a deliberately narrow, testable envelope:

- one controlled, well-lit room;
- one freestanding primary object with clear floor around it;
- a plain or slowly varying wall preferred behind the object;

- capture duration of approximately 30–120 seconds;
- base iPhone 17, without relying on rear LiDAR;
- a broad indexed furniture catalog plus a small synchronized set of injection-ready assets with validated USDZ and GLB derivatives;
- one active room session per warm GPU worker set until measured capacity supports greater concurrency.

The no-LiDAR constraint is load-bearing. All metric depth must come from ARKit visual-inertial odometry (poses, feature points, planes) combined with learned monocular or multi-view depth. This motivates both the conservative fast geometry track (Section 7.4) and the robust metric-alignment stage (Section 7.2).

2.2 Explicitly out of scope

The following are excluded from the current architecture: general-purpose removal of every object or non-planar background; whole-room automatic semantic discovery; recoloring a real object while preserving shading; multi-room mapping; cross-launch AR relocation; robust submap alignment after an ARKit world reset; on-device neural reconstruction or segmentation; dynamic people/pets as editable objects; photoreal Mode B1 on the critical path; and Android, visionOS, Quest, or OpenXR clients.

Each exclusion protects an invariant. Excluding on-device neural inference keeps the 60 Hz local loop deterministic and thermally bounded on a base iPhone 17. Excluding automatic submap alignment avoids silently rebasing committed edits onto a discontinuous world frame.

3 Architectural principles

3.1 Two parallel geometry tracks

Mode A does not wait for dense TSDF reconstruction before an object can be edited. The central structural decision in Reframe is the split into two parallel geometry tracks (Figure 2):

1. **Fast interaction track (critical path).** ARKit metric poses and planes, a reticle/tap-seeded SAM 3.1 video track, and a conservative multi-view silhouette volume. This produces an object identity, approximate volume, oriented bounding box (OBB), support relation, and reveal footprint quickly enough for replace/remove decisions.
2. **Dense understanding track (enhancement path).** Learned depth, robust metric alignment, and an Open3D VoxelBlockGrid TSDF. This improves dimensions, collisions, occlusion meshes, fallback-web geometry, and research metrics, but it may not block a valid fast-path edit.

The forbidden serial dependency is

learned depth → scale fit → TSDF → semantic lift → shell → compositor → first edit,

a chain whose end-to-end latency is the sum of its slowest stages and whose failure modes (per-view depth scale variation, fusion lag, extraction cost) would all surface as user-visible interaction failures. In the parallel design, the first edit depends only on ARKit’s already-metric pose and plane stream plus a 2D video track. The dense track upgrades artifacts asynchronously: dense results may improve dimensions and occluders but never invalidate committed object UUIDs or transactions.

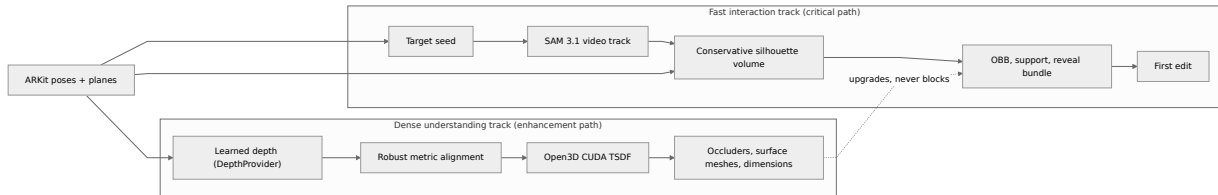


Figure 2: The two parallel geometry tracks. The fast interaction track builds an editable object identity from ARKit poses/planes and a SAM 3.1 video track without any learned depth. The dense understanding track runs asynchronously and upgrades artifacts; the dashed edge is non-blocking and never invalidates committed object UUIDs or transactions.

3.2 Permanent architecture rules

The following rules are permanent invariants of the architecture:

- **No camera-pixel punching in the primary compositor.** The camera background is untouched. Reframe overlays true-3D floor/wall reveal proxies and virtual assets. A Metal stencil compositor is the fallback when RealityKit cannot meet quality (Section 11.2).
- **A reveal may contain several surfaces.** A chair can expose floor, wall, and skirting board; the data model does not assume one plane (Section 8.2).
- **Readiness is capability-specific.** `replace_ready` and `remove_ready` are separate; replacement can become available before empty removal (Section 9).
- **The editing volume and collision surface are separate artifacts.** A conservative closed mask volume serves diminished reality; a denser surface mesh serves dimensions and collision.
- **Tap selection is first class.** Voice plus reticle is the primary interaction, but tap uses the same resolver and remains available without AI.
- **The GPU stack is separated into Docker services immediately.** LingBot/geometry and SAM 3.1 have incompatible recommended Python/PyTorch environments.
- **The depth model is isolated behind a provider interface.** DA3Metric and LingBot serve different input modes and must not leak provider state into scene identity.
- **Mode B0 is guaranteed.** Mode B1 never consumes live-path compute or blocks core product behavior.

3.3 Engineering decisions

Table 1 records the fourteen load-bearing technical decisions (RF-D1 to RF-D14) with their required choices and rationale. They are referenced throughout the report.

Table 1: Engineering decisions RF-D1 to RF-D14.

ID	Decision	Required choice	Reason
RF-D1	Native client	SwiftUI, ARKit, RealityKit/Metal	Direct spatial, audio, capture, and rendering control
RF-D2	Universal client	Separate Next.js application	Cross-device upload, replay, typed control, Mode B viewing
RF-D3	Metric pose	ARKit VIO on healthy native sessions	Low-latency, metric, gravity-aligned tracking
RF-D4	Live dependency	Fast semantic/plane path does not wait for TSDF	Keeps learned dense geometry off the interaction critical path
RF-D5	Dense geometry	DA3Metric-Large behind Open3D CUDA DepthProvider; TSDF	Replaceable inference with deterministic metric alignment and fusion
RF-D6	RGB-only mapping	LingBot-Map	Streaming trajectory and geometry for ordinary video
RF-D7	Semantics	SAM 3.1 point/box-seeded tracking	Persistent target identity without whole-room discovery
RF-D8	Diminished reality	Multi-surface reveal proxies	Keeps the real camera background intact
RF-D9	Rendering	RealityKit first; ARKit + Metal fallback	Native anchoring with an explicit compositor escape hatch
RF-D10	Agent	GPT Realtime 2.1 over WebRTC + gateway validation + GPT-5.6 Sol	Conversational input with schema-constrained planning
RF-D11	Catalog	Authorized acquisition + content-addressed assets + Qdrant	Scalable retrieval without large binaries in Git
RF-D12	Persistence	SQLite WAL + filesystem/shared volumes	Durable single-authority state without unnecessary infrastructure
RF-D13	Contracts	JSON Schema with Swift, TypeScript, and Python adapters	One wire definition across runtimes
RF-D14	Confirmation	Preview, then spoken or tapped confirmation	The model cannot directly mutate canonical scene state

4 System architecture and compute loops

4.1 End-to-end architecture

Figure 3 shows the complete system: the native iPhone application with its 60 Hz local loop; the gateway container that owns sessions, authentication, scene revisions, transactions, the agent boundary, and the catalog; three GPU worker

classes (geometry, semantics, reveal); and the Next.js web application. Four distinct compute loops run over this topology and are detailed in Sections 4.2–4.5.

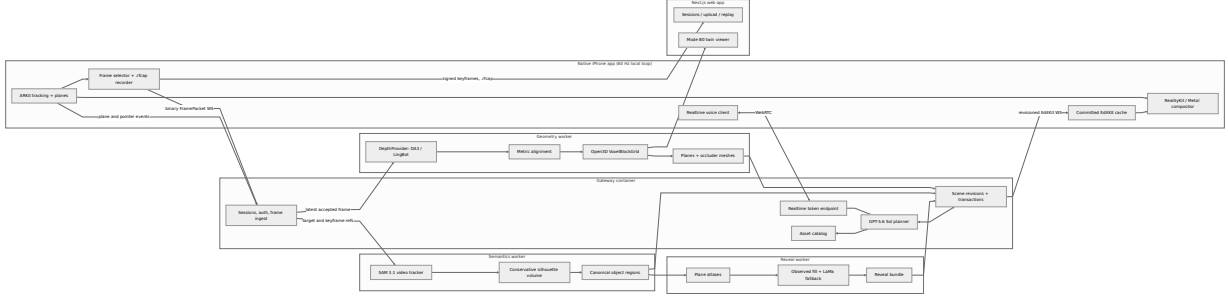


Figure 3: End-to-end Reframe architecture: native iPhone client, gateway container, geometry/semantics/reveal GPU workers, and the Next.js web application, with the principal data paths between them. The frame ingest socket carries one binary FramePacket stream; the gateway fans out only changed, revised EditKit artifacts back to clients.

4.2 The 60 Hz local loop

The local loop is independent of the network and targets 60 FPS (Figure 4):

1. ARKit updates camera pose, plane anchors, tracking quality, light estimate, and raycasts.
2. RealityKit or Metal draws the camera background.
3. The compositor draws retained-scene occlusion meshes.
4. For removed or replaced objects, it draws the active reveal layers.
5. It draws replacement and placed USDZ entities, contact shadows, and status UI.
6. Committed EditKit artifacts are stored locally and continue rendering if the network fails.

No per-frame neural model runs on the phone in the core release. Network failure degrades understanding, never committed rendering: new understanding pauses when disconnected, while committed edits remain visible.

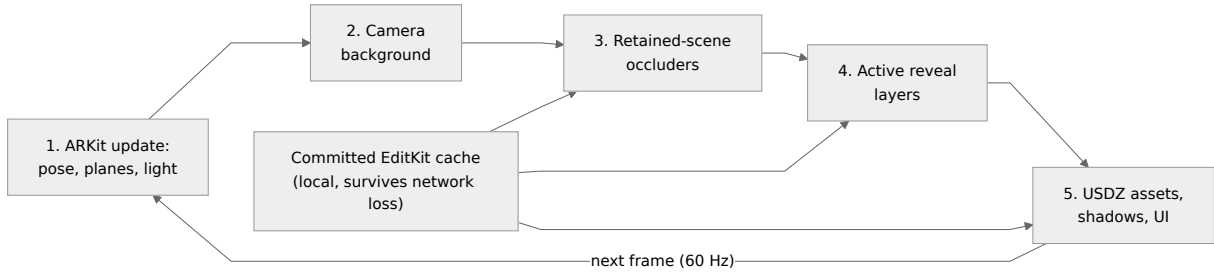


Figure 4: The 60 Hz local rendering loop. Rendering order is fixed: camera background, retained occluders, reveal layers, virtual assets and UI. All inputs to steps 3–5 come from the locally cached, committed EditKit artifacts, so the loop never blocks on the network.

4.3 The cloud fast-interaction loop

The fast interaction loop is event-driven (Figure 5):

1. Reticle dwell or tap captures `frame_id`, pixel, world ray, and ARKit hit.
2. SAM 3.1 starts or refines one video track.
3. Tracked masks and ARKit camera matrices form a conservative closed object volume.

4. ARKit planes establish floor/wall support and the approximate reveal surfaces.
5. Plane atlases and a reveal bundle are generated.
6. Capability states transition independently: `tracked`, `geometry_ready`, `replace_ready`, `remove_ready`.
7. The phone receives only changed, revisioned artifacts.

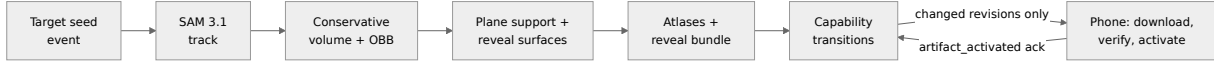


Figure 5: The event-driven cloud fast-interaction loop, from a `target_seed` event to independent capability transitions. The phone acknowledges each activated artifact revision; user-facing readiness requires that acknowledgement (Section 9).

4.4 The cloud dense-understanding loop

Accepted frames are processed asynchronously (Figure 6):

1. The DepthProvider predicts metric depth or reconstructs depth and pose.
2. Native ARKit sessions use ARKit as the pose authority.
3. Depth is converted into the project depth convention and aligned to ARKit-validated input (Section 7.2).
4. High-confidence measurements are integrated into a sparse CUDA TSDF.
5. The TSDF produces a retained-scene occluder mesh, object surface meshes, dimensions, and Mode B0 geometry.
6. Dense results may upgrade artifacts but never invalidate committed object UUIDs or transactions.

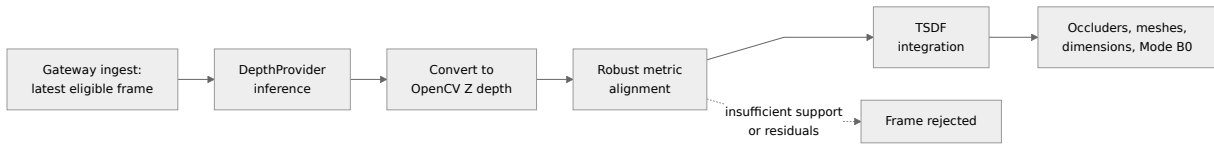


Figure 6: The asynchronous dense-understanding loop. Alignment may reject a frame outright; a plausible-looking but weakly aligned frame is worse than a dropped frame. Dense outputs upgrade existing artifacts and never create object identity on their own.

4.5 The voice/agent loop

The voice loop (Figure 7) is deliberately split across two model boundaries:

1. The iPhone obtains a short-lived Realtime credential from the gateway; the standard OpenAI API key never reaches the phone.
2. Full-duplex audio travels directly over WebRTC between the phone and OpenAI GPT Realtime 2.1 (`gpt-realtime-2.1`).
3. Realtime emits a minimal `submit_user_turn` function call, or asks a clarifying question, and the gateway schema-validates it.
4. The gateway invokes GPT-5.6 Sol through the Responses API with strict tool schemas for load-bearing intent and candidate selection.
5. Deterministic tools resolve spatial targets and validate physical constraints.
6. A preview is issued; commit creates a new scene revision and an EditKit delta.

Realtime is not treated as the source of canonical structured state: GPT Realtime 2.1 supports function calling but not Structured Outputs. Strictness and retries therefore live at the gateway/Sol planning boundary, not in the audio model. Typed commands enter immediately after `submit_user_turn` and invoke the exact same Sol/proposal pipeline, so voice is an input modality, never a separate editing system.

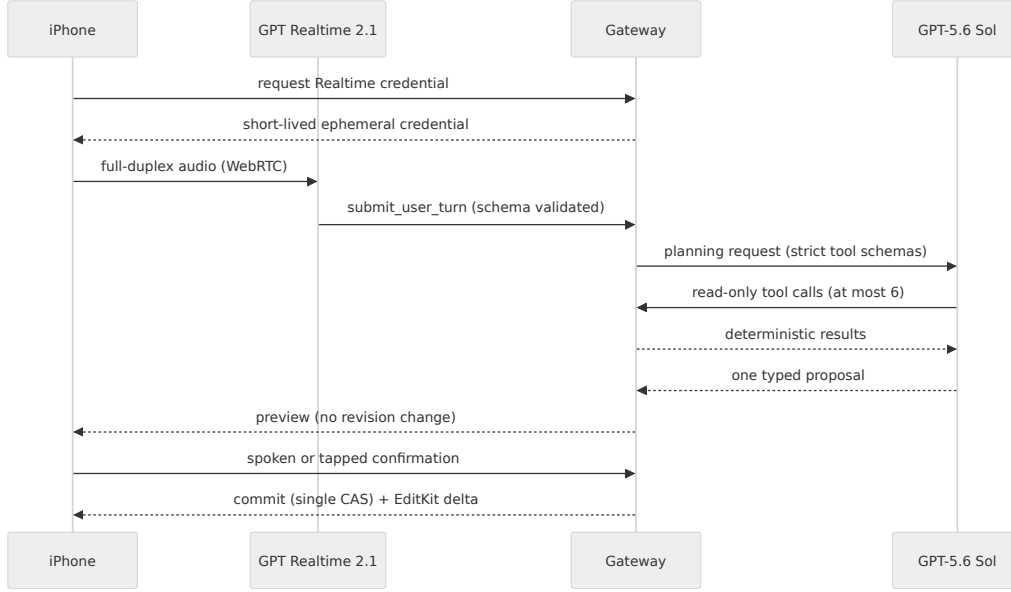


Figure 7: The voice/agent loop. Audio flows between the phone and GPT Realtime 2.1 over WebRTC; the only function Realtime may call is the non-mutating `submit_user_turn` ingress. Planning strictness lives at the gateway/GPT-5.6 Sol boundary, and commit requires explicit user confirmation followed by a single compare-and-swap transaction.

5 Coordinate, image, and time conventions

These conventions are invariants, not implementation suggestions. Every runtime (Swift, TypeScript, Python) implements them from the same normative definitions; the repository’s protocol package exposes them as code (RF-COORD-1) with unit tests.

5.1 World and camera coordinate frames

The world frame is right-handed, in metres, gravity aligned, with **+Y up**. ARKit camera local axes are **+X right**, **+Y up**, and the camera looks along **−Z**. Server/OpenCV camera axes are **+X right**, **+Y down**, **+Z forward**. Mathematics uses column vectors; serialization is row-major only. The axis-flip matrix between the two camera conventions is

$$C_{\text{arkit} \leftarrow \text{opencv}} = \text{diag}(1, -1, -1, 1) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (1)$$

which is numerically self-inverse ($C_{\text{arkit} \leftarrow \text{opencv}} = C_{\text{opencv} \leftarrow \text{arkit}}$). Its direction is nevertheless always named explicitly in code and on the wire to prevent silent double application. The conversion chain is

$$T_{\text{world} \leftarrow \text{cam}, \text{cv}} = T_{\text{world} \leftarrow \text{cam}, \text{arkit}} \cdot C_{\text{arkit} \leftarrow \text{opencv}}, \quad (2)$$

$$T_{\text{cam}, \text{cv} \leftarrow \text{world}} = (T_{\text{world} \leftarrow \text{cam}, \text{cv}})^{-1}. \quad (3)$$

`world_from_camera_arkit` is serialized by logical rows; a column-major `simd_float4x4` is never copied byte-for-byte into the wire buffer. Open3D integration receives the explicitly documented $T_{\text{cam}, \text{cv} \leftarrow \text{world}}$ extrinsic. Every matrix field carries a convention/version tag in the session manifest. Figure 8 illustrates the frames and the flip.

5.2 Image transforms and intrinsics

The encoded image is always orientation up and normalized to sRGB. Native bi-planar YCbCr conversion, crop, rotation, and resize are defined as one pixel transform A . If the native sensor image is transformed, the intrinsics transform identically:

$$K_{\text{encoded}} = A \cdot K_{\text{sensor}}. \quad (4)$$

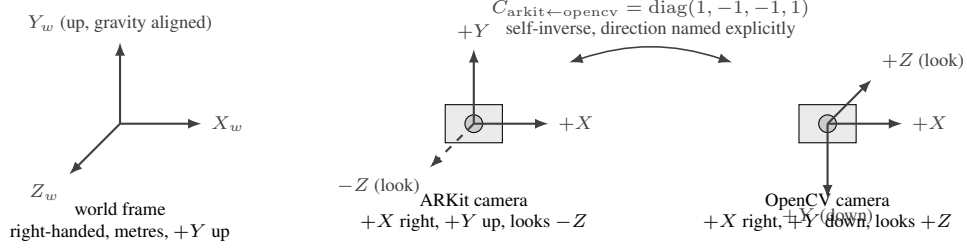


Figure 8: Coordinate frames: the gravity-aligned +Y-up world frame, the ARKit camera frame (looks along $-Z$), and the OpenCV camera frame (looks along $+Z$). The axis flip of Equation (1) converts between the camera conventions; because it is numerically self-inverse, its direction is always named to prevent double application.

The same transform must be applied to masks, point prompts, and returned pixels, so a pixel coordinate is meaningful only relative to a specific encoded image. A known-ray projection test is mandatory in continuous integration. Figure 9 shows the relationship.

The phone additionally emits two image products from the same ARFrame (Section 11.1); each carries its own transformed intrinsics, and low- and high-resolution images must never share an intrinsic matrix implicitly.

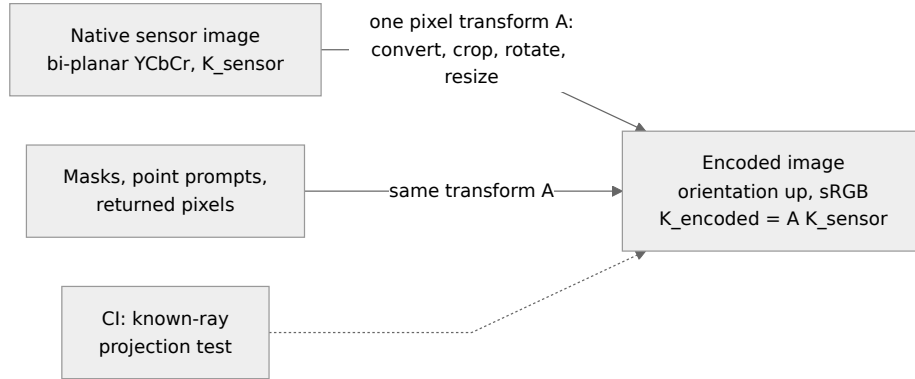


Figure 9: Image transform and intrinsics relationship. All pixel-space products (masks, prompts, returned pixels) live in the encoded image, whose intrinsics are $K_{\text{encoded}} = A K_{\text{sensor}}$ (Equation (4)); a known-ray projection test guards the convention in CI.

5.3 Depth convention

`DepthObservation.depth_m` is optical-axis/projective Z **depth in metres in the encoded OpenCV camera frame**, not Euclidean ray range. For a camera-frame point (X, Y, Z) the stored value is Z , related to the ray range r by

$$d_{\text{stored}} = Z = \frac{r}{\sqrt{1 + \tilde{x}^2 + \tilde{y}^2}}, \quad \tilde{x} = \frac{u - c_x}{f_x}, \quad \tilde{y} = \frac{v - c_y}{f_y}, \quad (5)$$

where (u, v) is the encoded pixel and (f_x, f_y, c_x, c_y) come from K_{encoded} . Providers must convert before returning. Confidence is normalized to $[0, 1]$, and provider-specific raw diagnostics are kept separately from the normalized observation.

5.4 Time and clock domains

`timestamp_ns` uses the iOS monotonic uptime clock derived from the AR frame timestamp, and `clock_domain` is explicit on every packet (the protocol also admits `browser_monotonic_performance` for the browser capture path). Network and server timings use monotonic clocks. The gateway performs periodic ping exchanges and records a clock-offset estimate; latency metrics never subtract unrelated wall clocks.

6 Wire and storage contracts

6.1 FramePacket: the binary frame contract

All live frames travel over a single binary WebSocket as FramePackets. All integers are little-endian. The packet is a fixed 24-byte header, followed by UTF-8 JSON metadata, followed by the image payload (JPEG in the core release). Figure 10 and Table 2 give the layout.

Table 2: FramePacket fixed 24-byte header layout (all integers little-endian).

Offset	Size	Field	Type
0	4	ASCII magic RFFP	bytes
4	2	protocol_version	uint16
6	2	flags	uint16
8	4	JSON metadata length	uint32
12	4	image payload length	uint32
16	8	frame_id	uint64

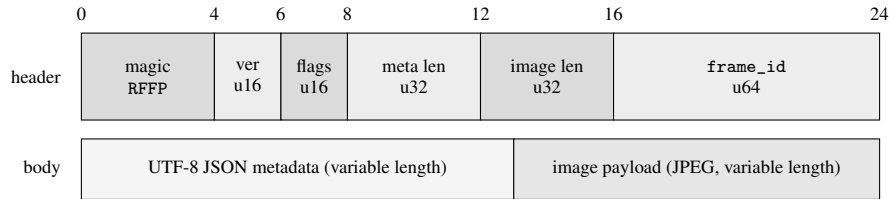


Figure 10: FramePacket binary layout: a fixed 24-byte little-endian header (byte offsets above), then UTF-8 JSON metadata, then the image payload. Unknown flags are ignored only when the protocol major version matches.

The JSON metadata carries the session/submap/frame identity, the explicit clock domain, the encoded-image description (codec, size, orientation up, sRGB, payload bytes), the 3×3 row-major `intrinsics_encoded`, the row-major `world_from_camera_arkit` pose, the ARKit tracking state and world frame version, and capture-quality signals (blur score, angular velocity, translation and rotation since the last accepted frame, exposure, ISO):

```
{
  "protocol_version": 1,
  "session_id": "room_2026_07_13_01",
  "submap_id": 0,
  "frame_id": 842,
  "timestamp_ns": 1783918472391823,
  "clock_domain": "ios_monotonic_uptime",
  "image": {
    "codec": "jpeg",
    "width": 640,
    "height": 480,
    "orientation": "up",
    "color_space": "sRGB",
    "payload_bytes": 48372
  },
  "intrinsics_encoded": [
    514.4, 0.0, 319.8,
    0.0, 513.9, 239.6,
    0.0, 0.0, 1.0
  ],
  "world_from_camera_arkit": [
    ...16 row-major floats...
  ],
  "tracking": {
    "state": "normal",
    "reason": "none",
    "world_frame_version": 1
  },
  "capture_quality": {
    "blur_score": 0.08,
    "angular_velocity_rad_s": 0.19,
    "translation_since_last_m": 0.034,
    "rotation_since_last_deg": 3.2,
    "exposure_s": 0.0083,
    "iso": 142
  }
}
```

6.1.1 Header flags

Table 3 lists the defined flag bits. Unknown flags are ignored only when the protocol major version matches.

6.1.2 Acceptance and backpressure

Frame acceptance is threshold-driven and bounded on both ends of the socket:

Table 3: FramePacket header flag bits.

Bit	Meaning
0	selected_keyframe
1	replay_packet
2	final_packet
3	high_priority_target_view
4	optional_depth_payload_present (reserved; unused on base iPhone 17 core release)

- **Native selection thresholds (initial values):** translation ≥ 0.03 m, rotation $\geq 3^\circ$, coverage gain, a high-priority target view, or 250 ms since the last accepted frame.
- **Rejection:** ARKit tracking not normal, excessive blur or angular velocity, transitioning exposure, or an unacknowledged frame on the socket with a full queue.
- **Queue capacities:** native queue capacity 2; worker queue capacity 1–2.
- **Overload policy:** keep the newest useful frame; never accumulate latency.

On the server side, the gateway retains a 5-second/64-frame recent-frame ring for target semantics and replay, and forwards only the latest eligible frame to the geometry queue: the ring is retention, not a compute backlog (Section 12.4).

6.2 Plane and pointer event contracts

Plane anchors and user pointing are control events, not data duplicated into every frame. ARKit plane IDs are ephemeral observations; the gateway/geometry worker associates them with versioned canonical surfaces such as `floor_01`. A removed or merged ARKit anchor never silently changes an object’s canonical support surface.

A `plane_upsert` event carries the session and plane IDs, a monotonic per-anchor revision, the classification, a row-major `world_from_plane` transform, metric extents, local-XZ boundary vertices, and a confidence. `plane_remove` carries the same session/plane ID and the final revision.

The `target_seed` event is the single entry point for all target selection routes:

```
{
  "type": "target_seed",
  "session_id": "room_2026_07_13_01",
  "frame_id": 842,
  "pixel_encoded": [318.0, 251.0],
  "ray_world": {
    "origin": [1.42, 1.53, -2.18],
    "direction": [0.11, -0.18, -0.98]
  },
  "arkit_hit": {
    "surface_id": "arkit_plane_07",
    "position_world": [1.66, 0.01, -4.31]
  },
  "source": "reticle_dwell"
}
```

`source` is one of `reticle_dwell`, `tap`, `voice_capture`, or `debug_web`. Because every route emits the same contract, tap selection is structurally guaranteed to be first class: it exercises the identical resolver as voice and reticle and remains available without any AI in the loop.

6.3 The `.rfcap` durable capture contract

While recording, `.rfcap` is a directory; on clean finish it may be zipped without changing its internal paths:

```
session.rfcap/
manifest.json
frames/
  0000000000001.jpg
  0000000000002.jpg
keyframes/
  0000000000842.jpg
frames.jsonl
keyframes.jsonl
events.jsonl
checksums.json
```

6.3.1 Durability guarantees

The recorder’s write path is ordered so a crash can lose at most the last buffered frame and can never corrupt prior records (Figure 11):

1. Every accepted packet is enqueued to the local writer before it is offered to the network sender.
2. Each frame image is written to a temporary name and atomically renamed.
3. `frames.jsonl` is append-only; batching flushes is allowed.
4. `checksums.json` is finalized at session end.

Replay emits the same logical `FramePacket` sequence, preserving IDs, timestamps, poses, intrinsics, and event ordering, so a replayed session is equivalent input for every downstream worker.

`keyframes/` contains sparse high-resolution stills (target long edge ≤ 1440 px initially, typically 8–20 around the primary target) keyed to the same AR frame IDs; they are encoded off the render thread and uploaded over signed HTTP, never through the live `FramePacket` socket. `keyframes.jsonl` records the high-resolution image size, the exact encoded-image intrinsics, the crop/orientation transform, the source frame ID, sharpness, and the selection reason.

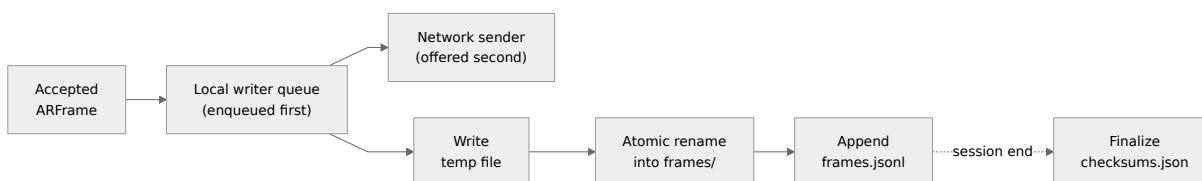


Figure 11: `.rfcap` atomic-write and durability sequence. Local durability is prioritized over network delivery: the writer queue is fed before the sender, images are renamed atomically, and the frame index is append-only. A crash loses at most the last buffered frame.

6.3.2 Manifest

The manifest pins the conventions the rest of the system depends on: format and version, session ID, creation time, device model and OS, world description (metres, right-handed, +Y up), image orientation up, `matrix_layout: row_major`, `camera_convention: arkit`, TSDF parameters (null until dense processing), and the user’s consent record including `retain_raw_until`.

7 Geometry: providers, alignment, fusion, and fast volumes

7.1 The DepthProvider contract

All learned depth sits behind one typed provider interface so that model choice never leaks into scene identity:

```
class DepthProvider(Protocol):
    def start_session(self, calibration: SessionCalibration) -> None: ...
    def infer(self, packet: FramePacket) -> DepthObservation: ...
    def finish_session(self) -> ProviderSummary: ...
```

A `DepthObservation` names the provider, declares `depth_convention: opencv_z_m`, references compressed depth and confidence payloads, states the pose source (arkit for native sessions), carries the explicit `world_from_camera_cv` extrinsic, and reports the alignment result (scale, bias, inliers, median absolute residual, accepted flag) plus per-stage timings. Table 4 summarizes the three provider variants.

A stable ARKit session never mixes LingBot poses into its world unless an explicit, measured alignment decision is recorded in the master prompt. A replay bake-off (the same `.rfcap` through providers A and B, and a video-only derivative through C) records median/p95 inference time, peak VRAM, floor-plane RMSE, taped-distance errors, TSDF wall thickness, edge quality around the primary object, temporal depth flicker, loop-return drift, rejected-frame rate, and a qualitative side-by-side mesh. The outcome is one bounded provider decision, not a general benchmark platform.

Table 4: DepthProvider implementations and their roles.

Provider	Input	Pose authority	Role and risk
DA3MetricProvider	single encoded frame + focal length	ARKit	Initial enhanced-path implementation; simple and metric; risk of per-view temporal scale/shape variation
DA3PoseConditionedProvider	short pose-conditioned DA3-Small/Base window	ARKit	Bake-off candidate; more temporally consistent multi-view geometry; window-management and latency complexity
LingBotProvider	RGB stream or video	LingBot (owns pose)	Universal fallback when ARKit metadata is absent or corrupt; never the default trajectory provider inside a healthy ARKit session

7.2 Robust metric alignment

Learned depth is metrically useful only after alignment against ARKit-validated measurements. For ARKit sessions the pipeline (Figure 12) is:

1. Convert provider output to OpenCV optical-axis Z depth (Equation (5)).
2. Project ARKit raw feature points and stable plane intersections into the encoded frame.
3. Sample predicted depths only at spatially distributed, valid correspondences.
4. Fit a robust scale-only correction first: $\hat{d}_i = s d_i^{\text{pred}}$, with s estimated robustly against the ARKit reference depths d_i^{ref} .
5. Permit a bounded affine bias, $\hat{d}_i = s d_i^{\text{pred}} + b$, only if it improves held-out residuals and has sufficient spatial support.
6. Temporally smooth accepted parameters with an exponential moving average.
7. Reject the frame if support or residual quality is insufficient.
8. Once the TSDF is stable, add TSDF raycast correspondences as additional validated input.

The initial tunable limits are: minimum inliers 20; scale clamp $s \in [0.85, 1.15]$; absolute bias clamp $|b| \leq 0.15$ m; minimum image quadrants with correspondences 3; EMA $\alpha = 0.10$. The governing principle is that a plausible-looking but weakly aligned frame is worse than a dropped frame: a silently mis-scaled frame corrupts the TSDF, whereas a dropped frame merely delays it.

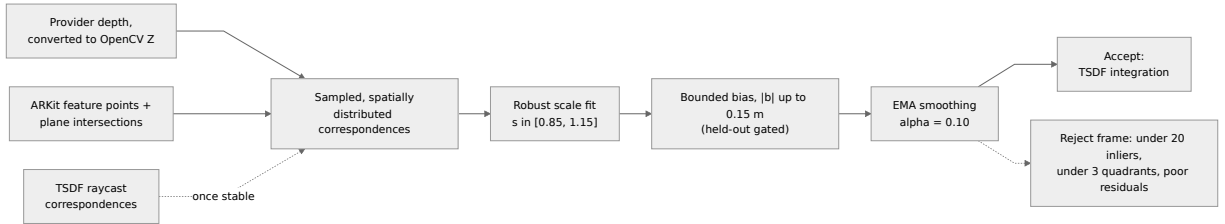


Figure 12: Robust metric-alignment pipeline. A scale-only correction is fitted first; a bounded affine bias is admitted only when it improves held-out residuals with sufficient spatial support; frames failing inlier, quadrant-coverage, or residual criteria are dropped rather than fused.

7.3 Dense TSDF fusion and canonical addressing

Dense fusion uses the Open3D tensor `VoxelBlockGrid` on CUDA with the initial configuration: voxel size 0.02 m, SDF truncation 0.08 m, depth range 0.25–6.0 m, maximum voxel weight 32. Integration follows the standard truncated-signed-distance update: for voxel v with current value $D(v)$ and weight $W(v)$ receiving a new truncated observation $d(v)$ with weight $w(v)$,

$$D(v) \leftarrow \frac{W(v) D(v) + w(v) d(v)}{W(v) + w(v)}, \quad W(v) \leftarrow \min(W(v) + w(v), W_{\max}), \quad W_{\max} = 32. \quad (6)$$

Integration is continuous, but mesh extraction and decimation run at a bounded cadence (initially 1 Hz) or on interaction demand; full-scene marching cubes never runs per input frame. Active blocks are limited to the observed room envelope, and evictions/reloads are recorded.

Before integration, each depth map is filtered: confidence is derived from provider output when available, otherwise from temporal agreement, alignment residuals, and edge proximity; low-confidence pixels are removed; depth discontinuities are eroded; gross disagreement with a stable raycast is rejected; target and object masks are excluded where dynamic motion is suspected; and integration requires normal ARKit tracking plus accepted depth alignment.

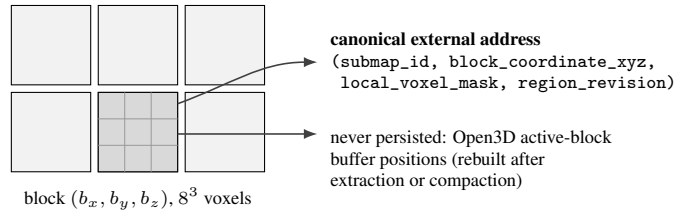


Figure 13: TSDF canonical addressing. External references name stable coordinates (submap, integer block coordinate, local voxel mask, and region revision), never Open3D’s internal active-block buffer positions, which are invalidated by extraction and compaction.

External references use the canonical address (submap_id, block_coordinate_xyz, local_voxel_mask, region_revision) (Figure 13); Open3D active-block buffer positions are never persisted, and mappings are rebuilt after extraction or compaction. TSDF outputs are: decimated retained-scene occluder chunks, object surface meshes for stable semantic regions, plane refinements, dimensions and collision proxies, the Mode B0 mesh/point representation, and raycasting for alignment and validation. Dense outputs upgrade existing objects; they do not create new object UUIDs without semantic confirmation.

7.4 Fast interaction geometry: conservative silhouette volumes

7.4.1 Target acquisition

The core release supports three acquisition routes: center-reticle dwell of approximately 400 ms, explicit tap, and an utterance-time reticle snapshot. All three generate the same `target_seed` contract (Section 6.2).

7.4.2 Volume construction

The fast path builds a closed volume without any learned depth (Figure 14):

1. Seed a SAM video track on the selected frame.
2. Collect two to four views with normal tracking and at least one useful baseline: ≥ 0.15 m translation or $\geq 12^\circ$ view change; prefer $20\text{--}60^\circ$ rather than nearly duplicate or opposite views. A target seed enables a bounded 1.5-second 10–12 FPS upload burst while stale frames are still dropped.
3. Establish a candidate 3D region of interest using support-plane intersection, the mask ray bundle, category-agnostic furniture height limits, and room bounds.
4. Discretize at 2–3 cm.
5. Project each voxel center into each valid mask.
6. Accumulate confidence from signed mask distance, view angle, track score, and image quality.
7. Keep voxels supported by at least two views and a weighted occupancy threshold; avoid strict all-view intersection.
8. Fill enclosed cavities, perform one-voxel closing, and dilate by approximately 3–4 cm.
9. Extract the OBB and an optional low-triangle boundary mesh.

The artifact is intentionally conservative: slight overcoverage is safer than leaving real-object fragments visible after removal or replacement. The same asymmetry drives the cover-score thresholds (Section 10.5).

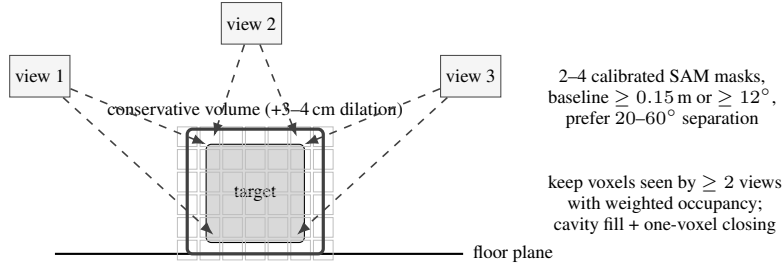


Figure 14: Conservative multi-view silhouette-volume carving. Voxel centers in a support-plane-anchored region of interest are projected into each calibrated SAM mask; occupancy is a weighted vote (not a strict all-view intersection), followed by cavity filling, morphological closing, and a 3–4 cm safety dilation.

7.4.3 Support and dependency

Floor support is inferred from the lowest occupied voxels and the stable ARKit floor plane. Tabletop or shelf support is deferred unless the dense geometry path already exposes a reliable support plane. Removing an object with dependents is blocked; the core release may offer a confirmed cascade only when all dependent objects are themselves virtual or have valid reveal coverage.

8 Scene representation and EditKit artifacts

8.1 The canonical scene graph

The canonical scene graph is the single authoritative description of the edited room. It contains: a `world_frame` descriptor (ID, version, metric flag, handedness, up axis, pose source, scale confidence); versioned canonical surfaces (for example `floor_01`, `wall_02`) with world plane equations, atlas references, and confidences; objects with per-capability readiness flags, advisory labels, references to spatial regions, mask volumes, surface meshes, OBBs, support-surface IDs, reveal-bundle references, and visibility state; and `placed_assets` instances with world transforms.

Identity rules are strict:

- The scene graph never contains RealityKit entity IDs, Three.js object indices, Gaussian IDs, or Open3D buffer indices; engine-local identifiers are non-canonical by construction.
- Object labels (category, attributes, confidence, source `gpt-5.6-sol`) are advisory; a label change never changes identity.
- ARKit plane anchors are ephemeral observations mapped onto versioned canonical surfaces; anchor churn never silently changes an object's support surface.
- Optional `navigation_constraints` may contain named 2D floor polygons (for example `main_walkway`); when absent, validation reports only a minimum geometric gap and must not claim full route connectivity.

8.2 EditKit object artifacts

Every external artifact reference carries `mime_type`, `byte_length`, and `sha256` in the normative JSON Schema. Clients verify length and hash before atomic activation; a failed artifact never advances the scene revision locally. After GPU/resource creation succeeds, the client sends `artifact_activated {artifact_id, revision, sha256}`; capability chips use this acknowledgement rather than server generation status alone.

Per object, three geometric artifacts serve distinct purposes:

- `mask_volume`: a conservative, closed, run-length-encoded voxel volume (voxel size 0.025 m initially, with an explicit `conservative_dilation_m`), used for diminished reality. It is not assumed to be an accurate visible surface.
- `mask_mesh`: the renderable isosurface of the mask volume, used by the Metal stencil fallback; the RealityKit plane-proxy path may not need it.
- `surface_mesh`: the less conservative TSDF-derived geometry used for collision and dimensions; it may be absent when the fast path first reaches `geometry_ready`.

This separation implements the rule that the editing volume and the collision surface are different artifacts with different accuracy and safety trade-offs. The object record also carries the OBB (world center, metric extents, yaw).

8.3 The reveal bundle

A reveal bundle is the multi-surface diminished-reality artifact for one object (Figure 15). `layers[] .geometry .type` is a tagged union: the core release implements `plane_polygon`; `textured_mesh` is permitted for a skirting board or another thin non-coplanar reveal without changing the bundle contract (the product still prefers floor/wall-backed targets). Each layer names its canonical surface, carries world-space polygon vertices with UVs, color and alpha texture URLs, a `reference_color_space` (linear sRGB), photometric reference frame IDs, a per-textel provenance map (`source_map_url`), a feather width (`feather_px_at_1080p`: 8), and a quality score. The bundle-level coverage block records the number of sampled views and the p10/median target coverage that gate `remove_ready` (Section 11.4).

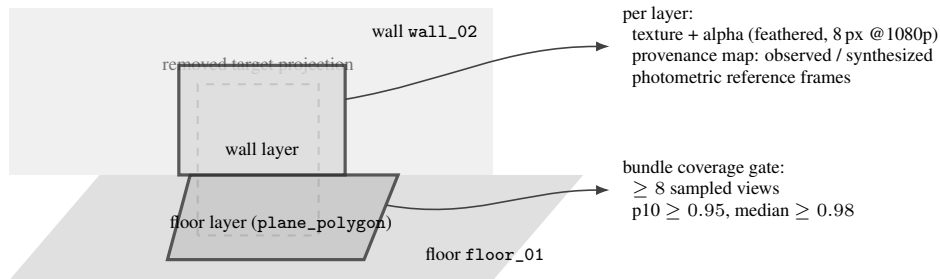


Figure 15: A multi-surface reveal bundle. One removed chair exposes two layers, a floor polygon and a wall polygon on their canonical surfaces, each with feathered color/alpha textures and per-textel provenance. Layer polygons are expanded to cover the target's projection across the captured primary view cone; modest overdraw of the same physical plane is preferred to incomplete coverage.

8.4 Occluder chunks and asset manifests

Retained-scene occlusion geometry ships as revisioned chunks. Each carries a `world_from_chunk` transform, a mesh URL, a triangle count, and `excluded_object_ids`: the mechanism by which a hidden target's occluder contribution is disabled while the rest of the room continues to occlude reveal surfaces and virtual furniture.

Catalog assets are described by a manifest with category, style, and color attributes; metric dimensions; a normalized origin (`floor_contact_center`) and fixed forward axis ($-Z$); a delivery class (for example `bundled_p0` with a local bundle key); signed USDZ and GLB URLs; a collision hull URL; the mobile download size; and a license record (SPDX identifier, source URL, attribution).

8.5 Edit deltas, inverse operations, and local undo

Scene mutations reach clients as idempotent `edit_delta` events:

```
{
  "type": "edit_delta",
  "scene_revision": 48,
  "base_scene_revision": 47,
  "transaction_id": "txn_081",
  "idempotency_key": "b02735d7-9204-4f22-8ef8-943f46793f11",
  "ops": [
    {"op": "set_object_visibility", "object_id": "chair_04",
     "value": "hidden"},
    {"op": "set_reveal_visibility",
     "reveal_bundle_id": "reveal_chair_04_v5", "value": true},
    {"op": "place_asset", "asset_id": "chair_red_02",
     "instance_id": "placed_017",
     "world_from_asset": ["16 float32"]}
  ],
  "inverse_ops": [
    {"op": "remove_asset_instance", "instance_id": "placed_017"},
    {"op": "set_reveal_visibility",
     "reveal_bundle_id": "reveal_chair_04_v5", "value": false},
    {"op": "set_object_visibility", "object_id": "chair_04",
     "value": "visible"}
  ],
}
```

```

"local_undo": {"token": "undo_txn_081",
               "valid_for_committed_revision": 48}
}

```

The phone acknowledges `scene_revision` and each artifact revision. During the active AR session the phone may apply `inverse_ops` immediately without the network, mark the undo `pending_sync`, and submit the undo token when connectivity returns; the gateway then creates the next canonical scene revision. The core release assumes one editor, so no concurrent merge is attempted.

9 Capability and lifecycle model

A single editable flag is insufficient for a system in which different edits have different evidence requirements. Each object exposes independent capability readiness through the lifecycle of Figure 16: `candidate` (observed once), `tracked` (stable 2D video identity), `geometry_ready` (conservative 3D volume, OBB, and support relation), `replace_ready` (geometry ready plus a validated asset candidate with sufficient visual cover), `remove_ready` (geometry ready plus a reveal bundle passing coverage and quality requirements), and `retired/uncertain` (contradictory, lost, dynamic, or unsupported).

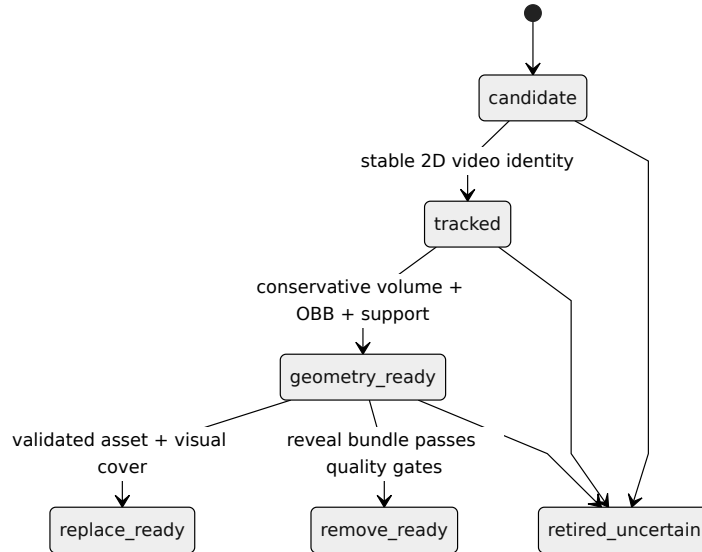


Figure 16: Capability lifecycle state machine. `replace_ready` and `remove_ready` branch independently from `geometry_ready`: replacement often becomes available before empty removal because a covering asset needs less reveal evidence than a fully exposed background. Any state may fall to `retired/uncertain`.

9.1 Readiness rules: server-validated $\cap$ client-activated

User-facing readiness is the intersection of server-validated input and client activation. A server artifact is not ready until the phone has downloaded it, verified its hash, created the render entity, and acknowledged the artifact revision. This closes the gap in which a server would advertise a capability whose supporting artifact the phone cannot yet render. Table 5 lists the minimum validated input per capability.

Table 5: Capability readiness: minimum validated input.

Capability	Minimum validated input
<code>place_ready</code>	stable ARKit floor plane and valid raycast
<code>tracked</code>	stable SAM identity; one view may establish the track
<code>geometry_ready</code>	at least two calibrated masks with adequate baseline, closed conservative volume, OBB, and support surface
<code>replace_ready</code>	geometry ready, asset fits, and asset-only or asset+reveal composite coverage passes
<code>remove_ready</code>	reveal bundle covers the projected target across sampled views and passes texture/seam checks

9.2 User-facing states

The interface communicates six states: *Looking* (no target selected); *Understanding* (target selected; track and volume being built); *Replace ready* (replacement is safe even if empty removal is not); *Remove ready* (reveal bundle meets quality thresholds); *Healing* (reveal generation still running; the system must not pretend removal is instant); and *Tracking paused* (ARKit tracking is limited; new edits are disabled but committed edits remain visible).

10 Transactions, tools, and the agent boundary

10.1 Transaction state machine

Every edit flows through one deterministic pipeline (Figure 17):

observe → resolve target → interpret constraints → retrieve candidates → propose
→ deterministic validation → preview → commit → undo/restore.

A transaction includes `base_scene_revision`, an `idempotency_key`, target-resolution confidence, constraints, validation results, and status. Mutating tools never run in parallel. Preview never changes the scene revision; commit is a single compare-and-swap (CAS) against `base_scene_revision`; committed history is immutable, and undo/restore appends an inverse transaction rather than rewriting history.

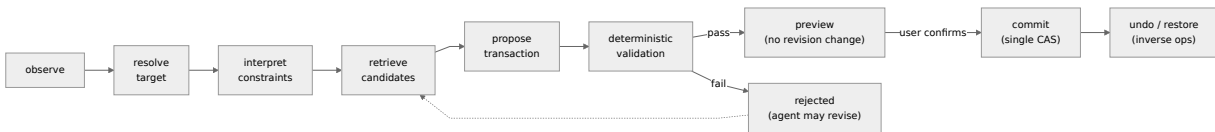


Figure 17: Transaction state machine. Validation failure returns a typed rejection the agent may react to (it cannot override geometry); preview never advances the scene revision; commit is one compare-and-swap; undo appends inverse operations and can be applied locally without the network.

10.2 Realtime ingress: submit_user_turn

GPT Realtime 2.1 may invoke only one narrow, non-mutating ingress function:

```
{
  "name": "submit_user_turn",
  "arguments": {
    "client_turn_id": "turn_019",
    "utterance": "Replace this chair with something warmer and red,
                  but keep the walkway clear.",
    "intent_hint": "replace",
    "pointer_context_id": "ptr_842_01",
    "client_scene_revision": 48,
    "pending_proposal_id": null
  }
}
```

The gateway validates, rate-limits, deduplicates, and attaches authoritative scene and pointer context. Two properties are invariant: Realtime cannot call mutating scene tools directly, and a typed turn carries only an opaque `pointer_context_id`, never a client-supplied world position. The gateway resolves that identifier against its own durable pointer state, so a compromised or buggy client cannot inject fabricated spatial authority into planning.

10.3 Public GPT tools

GPT-5.6 Sol sees exactly five strict, read-only/preview tools, all with `additionalProperties: false` JSON Schemas:

```
get_scene_context(region?, detail_level?)
resolve_target(pointer_context_id?, language_reference?)
search_catalog(category?, style?, color?, material?,
               max_dimensions_m?, budget?)
validate_candidate(target_id, asset_id, constraints?)
prepare_edit_preview(intent, target_id?, asset_id?, constraints?)
```

Commit, restore, hide-object, show-reveal, set-transform, and change-visibility are internal transaction-service functions, not agent-facing primitives. A turn may use at most six tool calls, inspect at most eight validated candidates, and produce at most one pending proposal. Sol never estimates 3D coordinates and never confirms an edit itself.

10.4 Deterministic validation

Validation is owned entirely by deterministic code and checks: target capability readiness; support surface; asset dimensions and scale; bundled/cache availability for core-release candidates; collision hull; room boundary; wall intersection; 2D minimum floor gap or an explicitly defined walkway-corridor clearance; scene-revision freshness; dependent supported objects; reveal readiness; and the replacement visual cover score. The agent may react to a rejection but cannot override geometry.

Clearance is an honest 2D floor-footprint calculation, not path planning: collision hulls and OBBs are projected onto the support plane, distances to room-boundary segments and retained obstacle footprints are computed, and the minimum free gap is reported. A spoken “keep the walkway clear” constraint maps to `minimum_gap_m` unless a named walkway corridor exists in `navigation_constraints`.

10.5 The replacement cover score

Replacement believability is gated by a deterministic silhouette-coverage score. For sampled captured viewpoints v around the target, the opaque delivered replacement geometry is rasterized (not its convex hull, and including material cutouts) and compared against the old target’s projection T_v :

$$\text{cover}_{\text{asset}}(v) = \frac{\text{area}(T_v \cap S_v^{\text{asset}})}{\text{area}(T_v)}, \quad (7)$$

$$\text{cover}_{\text{composite}}(v) = \frac{\text{area}(T_v \cap (S_v^{\text{asset}} \cup R_v))}{\text{area}(T_v)}, \quad (8)$$

where S_v^{asset} is the opaque asset silhouette and R_v the projection of the active reveal layers in view v (Figure 18). The core-release acceptance thresholds are:

- asset-only: median $\text{cover}_{\text{asset}} \geq 0.98$ and tenth percentile ≥ 0.95 ; **or**
- composite: median $\text{cover}_{\text{composite}} \geq 0.995$, tenth percentile ≥ 0.98 , and no uncovered connected component larger than 1% of the original target projection.

An asset-only score below threshold may be used only when the required reveal layers are ready and the combined score passes. The thresholds deliberately favor similarly sized or slightly larger replacements and prevent small fragments of the real object leaking around legs or armrests. Real products are never semantically rescaled after unit normalization, and the candidate search explores only bounded translation and yaw around floor-contact-center alignment.

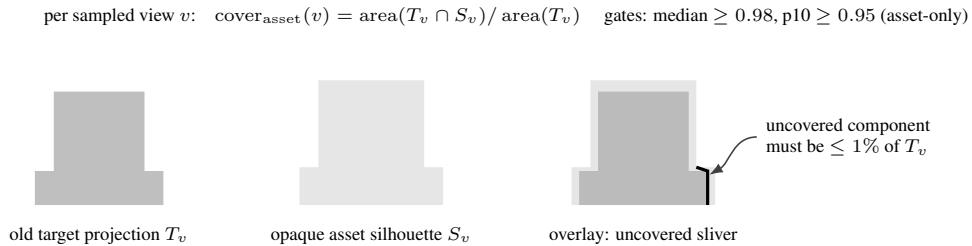


Figure 18: Cover-score silhouette intersection. The exact delivered opaque geometry is rasterized per sampled view and intersected with the old target’s projection; residual slivers around legs or armrests either fail the gate or must be covered by ready reveal layers under the composite thresholds of Section 10.5.

10.6 Failure behavior

Voice unavailable: typed or tapped interaction drives the same edit pipeline. Sol timeout: return a bounded failure and preserve any current preview; never commit automatically. Ambiguous target: ask one concise disambiguation, naming candidates. Stale revision: revalidate automatically, then ask only if intent materially changes.

11 Subsystem specifications

11.1 Native iOS capture application

The client is a SwiftUI application shell over an ARSession with ARWorldTrackingConfiguration: gravity-aligned world tracking, horizontal and vertical plane detection, autofocus enabled, and a camera format chosen to sustain stable tracking and at least 30 captured frames/s (the app renders at display cadence). Expensive frame semantics are not enabled without a measured need. ARCamera.TrackingState and its reason are monitored continuously. The compositor uses RealityKit with a Metal fallback; networking uses native URLSessionWebSocketTask/Network-framework WebSockets. No Capacitor, Flutter, Unity, or WebView appears in the hot path.

11.1.1 Frame selection

The phone emits two image products from the same ARFrame: low-resolution live packets (initially 640×480 JPEG) over the bounded WebSocket, and sparse high-resolution keyframes (long edge ≤ 1440 px, typically 8–20 around the primary target) written to .rfcap and uploaded asynchronously over signed HTTP for reveal textures, naming crops, Mode B0, and optional Mode B1. Encoding and disk I/O run off the render thread. Each product keeps its own transformed intrinsics (Section 5.2). Selection thresholds are configuration, not constants scattered through Swift:

```
min_translation_m: 0.03
min_rotation_deg: 3.0
max_interval_ms: 250
max_pending_packets: 2
max_blur_score: 0.35
max_angular_velocity_rad_s: 1.2
```

A target_seed event temporarily promotes useful new viewpoints around that object (the bounded burst of Section 7.4).

11.1.2 Tracking loss

Core-release behavior: on limited tracking, pause frame acceptance, keep local AR rendering, and show coaching; if tracking recovers in the same world frame, resume; on a world reset or discontinuity, stop live cloud integration, preserve .rfcap, and offer restart or Mode B0 processing. Automatic server-side submap alignment is explicitly not implemented.

11.1.3 Local persistence

At commit time the phone caches the reveal bundle textures and geometry, target object state, the USDZ asset and transform, the occluder chunks required by the edit, and the latest committed scene revision. Offline persistence is guaranteed for the active AR session; cross-launch relocalization is not promised.

11.2 On-phone compositor

11.2.1 Primary RealityKit render model

The RealityKit compositor renders, in order: (1) the AR camera background; (2) retained-scene occluder entities using an occlusion-only material; (3) reveal-layer entities with unlit textures, alpha feather, and a small depth bias; (4) replacement and placed USDZ entities; and (5) the contact-shadow blob, ambient tint and light estimate, readiness UI, reticle, and coaching. Until the dense retained-scene occluder arrives, the core release uses ARKit planes, and the staged room avoids foreground blockers across the primary walk path; the product must not imply general real-object occlusion before the proxy mesh is available. A hidden target's occluder contribution is disabled while other room geometry continues to occlude reveal surfaces and virtual furniture.

11.2.2 Why no primary screen-space punch-out

The camera feed is immutable. A reveal layer is actual virtual background geometry at the floor/wall location, so the same content reprojects correctly as the camera moves. Large observed-texture areas may extend beyond the old silhouette; because they represent the same physical plane, this is preferable to unstable per-frame mask cutting. Boundaries are feathered and selected in low-gradient areas.

11.2.3 Metal stencil fallback criteria

RealityKit verification is timeboxed to half a day. The compositor switches to ARKit + Metal when any of the following cannot be demonstrated on canned data: deterministic layer ordering; reveal geometry overlaying camera content correctly; retained-scene occlusion; coherent USDZ/mesh asset occlusion; stable 45+ FPS on the iPhone 17; or absence of severe alpha/depth artifacts at reveal boundaries. The Metal fallback may render a projected mask volume into stencil and composite reveal textures explicitly; the `mask_mesh` artifact exists for this path.

11.2.4 Revision behavior and photometric adaptation

Artifact revisions are downloaded before activation and crossfaded over 150–250 ms. At 3–5 Hz, the compositor estimates a smoothed per-channel gain and small luma offset from a visible ring of the same plane around the reveal and applies it in the reveal material, so the cached atlas follows current exposure and white balance; it falls back to the ARKit light estimate when the ring is insufficient. Reveal and mask-volume revisions freeze after commit unless improvement exceeds a logged quality threshold; a new occluder revision is swapped atomically.

11.3 Semantics and identity: SAM 3.1

The semantics worker is a separate Python 3.12+ container running the current SAM 3.1 code and checkpoint with point/box prompts derived from `target_seed`. One primary track is the core release; background concept discovery is out of scope. The tracker maintains per-session state, prioritizes target-request jobs over every background task, keeps frame IDs and encoded pixel coordinates exact, stores masks as compressed RLE with scores, and requests additional views when track confidence or baseline is insufficient.

When depth or TSDF data is available, canonical association proceeds by projecting canonical surfaces into tracked keyframes, verifying expected visibility using observed depth, accumulating object probability (down-weighting mask borders, grazing views, low confidence, and inconsistent depth), applying connected components and support-plane priors, and persisting a canonical region reference.

After track stabilization, one structured GPT-5.6 Sol request, containing two to four crops, approximate dimensions, the support relation, a candidate category list, and the object UUID, produces an advisory label. A label change never changes identity.

11.4 Plane atlases and reveal generation

11.4.1 Plane atlases

Each stable plane defines a world origin, an orthonormal U/V basis, metric atlas bounds, a color atlas, an observed-weight map, a source-frame/origin map, and a synthesized mask. For every useful frame: candidate pixels are ray-intersected with the plane and required to lie within the current plane boundary; target masks and known foreground object masks are excluded; when dense depth or a stable TSDF raycast exists, pixels whose observed depth is in front of the plane are rejected (before that exists, multi-frame photometric consistency is required and core-release atlases are restricted to the staged primary floor and wall); contributions are weighted by view angle, blur, distance, exposure consistency, and tracking quality; blending happens in linear color space with robust outlier rejection; and observed texels are permanently preferred over synthesized texels. An ARKit plane anchor is a geometric prior, not proof that every pixel sees that plane; foreground rejection is therefore mandatory.

11.4.2 Fill hierarchy

Reveal texture content follows a strict three-level hierarchy (Figure 19): (1) real observed atlas samples; (2) deterministic local texture copying and inpainting for simple low-gradient regions; (3) a LaMa fallback in its isolated worker. No diffusion inpainting ships in the initial release. Every texel is tagged `observed(weight, frame_id)` or `synthesized(method, revision)`; observed data may replace synthesized data, synthesized data never replaces observed data; updates are discrete and crossfaded; and after commit the bundle freezes unless an objective seam/coverage score improves materially.

11.4.3 Readiness quality gates

At least eight camera poses are sampled from the captured trajectory around the target; the conservative target volume and active reveal layers are projected into each. `remove_ready` requires: p10 target coverage ≥ 0.95 ; median target coverage ≥ 0.98 ; no uncovered connected component larger than 1% of the target projection; and an atlas observed

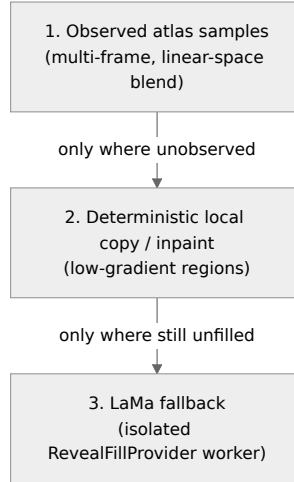


Figure 19: Reveal fill hierarchy. Observed content always wins; deterministic fills cover simple unobserved regions; the learned LaMa provider is a last resort behind an isolated provider boundary, and diffusion inpainting is excluded from the initial release.

fraction outside the synthesized hole of ≥ 0.80 . If the gate fails, replacement may still be ready while empty removal stays disabled; this is the capability split of Section 9.

Runtime readiness is fully automatic and contains no fixture, evidence-record, or human-vote fields. A physical half-circle visual review remains a release acceptance test for the compositor and reveal pipeline; it is not repeated per user session. When the camera leaves a reveal’s validated view envelope, the phone disables that reveal and its replacement together, shows the untouched camera feed, and asks the user to return to the captured area; this local safety behavior does not change canonical scene state.

11.5 Catalog: acquisition, preparation, retrieval, delivery

11.5.1 Acquisition and preparation lifecycle

The catalog pipeline discovers the authorized US-English IKEA product frontier, observes product-model requests in a controlled browser, and downloads every discoverable GLB with bounded concurrency, retry/backoff, checkpointing, ETag support, SHA-256 deduplication, and resumable state. Raw and derived binaries live in a content-addressed persistent volume, never in Git. For every asset the pipeline retains the canonical product URL, market, locale, source metadata, and authorization status; validates the GLB; normalizes units to metres, a floor-contact-center origin, and a fixed forward axis; records explicit dimensions and a low-poly collision proxy; produces a compressed web GLB and a compliant mobile USDZ derivative, plus bounded textures, mobile LODs, and a canonical turntable image; records immutable hashes for every input and output; and enriches category, color, style, material, and descriptive labels from product data and GPT-5.6 vision. Figure 20 shows the resulting per-asset lifecycle from discovered to injection_ready.

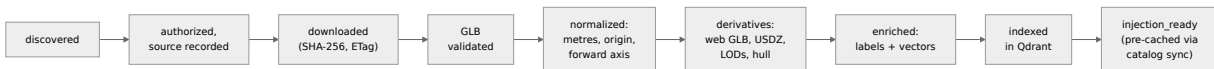


Figure 20: Catalog asset lifecycle. Every stage records immutable hashes; the complete corpus stays in persistent catalog and Qdrant volumes, and an explicit catalog sync -profile primary operation pre-caches the small injection-ready set. Application builds never download assets and fail clearly if the requested cache is missing.

11.5.2 Qdrant schema and hybrid retrieval

Qdrant stores metadata filters and the 1,024-dimensional semantic_v1 named vector generated from text plus visual descriptors; the schema reserves visual_v1 for a later local image encoder. Retrieval order is strict (Figure 21):

deterministic eligibility filters first (category, dimensions, support type, cache profile), vector ranking second, and GPT reranking only over the bounded eligible set. Qdrant is dedicated to catalog retrieval and never becomes scene or transaction authority.

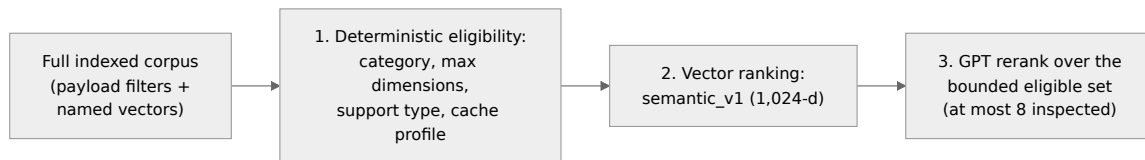


Figure 21: Qdrant schema and hybrid retrieval order: deterministic filters bound the candidate set before any vector similarity, and language-model reranking sees only that bounded, already-eligible set.

11.6 Placement, replacement, removal, and restore semantics

Placement. A local ghost appears immediately using an ARKit raycast; server validation checks support, bounds, collision, wall intersection, orientation, and clearance; a corrected transform is smoothly reconciled, never teleported without indication; a soft contact-shadow blob and ARKit light-estimate tint complete the presentation.

Replacement. Resolve the real target; retrieve candidates by dimensions and style; validate geometry and cover score (Section 10.5); preview locally as reveal plus asset; require spoken or tapped confirmation; commit as one CAS transaction.

Removal. Only when `remove_ready`: hide the target, activate the reveal bundle, and disable the target’s occluder contribution.

Restore/undo. Restores target visibility, hides reveal and replacement, and needs no server call (Section 8.5).

11.7 Gateway, GPT-5.6 Sol, and voice

The gateway owns: session lifecycle and room-scoped tokens; authenticated single-upload `FramePacket` ingest with the 5-second/64-frame recent-frame ring, a separate latest-only compute queue, and internal frame fan-out; scene-revision authority; the transaction state machine; strict tool endpoint implementations; GPT-5.6 Sol calls; the Realtime session credential endpoint; asset retrieval; WebSocket artifact fan-out; and SQLite-WAL metadata with an append-only transaction log.

Load-bearing Sol behavior for replacement: interpret category, style, and color constraints; understand the selected object and its attributes; choose among multiple fitting catalog candidates; react to a deterministic rejection; and explain the selected candidate and clearance result. For the example *“Replace this chair with something warmer and red, but keep the walkway clear,”* Sol receives the resolved target and deterministic room facts, searches the eligible catalog, receives fit rejection or acceptance, revises if necessary, and prepares one typed preview. It never estimates 3D coordinates and never confirms the edit itself.

The audio fallback streams PCM from `AVAudioEngine` to the gateway, which uses the Realtime server WebSocket; this changes transport only, not tools or transactions.

11.8 Next.js web application and Mode B0

11.8.1 Routes and stack

/	landing and project explanation
/sessions	session list/create/upload
/capture	browser video fallback
/replay/[sessionId]	synchronized image/pose/event replay
/twin/[sessionId]	mesh/point twin and edits
/catalog	searchable asset catalog and readiness

The stack is Next.js + TypeScript, Three.js for TSDF mesh/point rendering, Zod-generated contract validators, WebSockets for session events and edits, and regular HTTP upload for `.rfcap` and video. Spark is installed only if Mode B1 starts. The web application works on modern desktop and mobile browsers for upload, replay, viewing, and control; it does not promise the live AR compositor outside the native iPhone application in v1.

For `.rfcap` input, ARKit poses are reused; for ordinary uploaded video, LingBot-Map owns the camera trajectory and geometry, and the ingested stream carries no ARKit authority. The repository explicitly removed a code path that fabricated ARKit-authority metadata for browser video. Mode B0 shares scene, transaction, catalog, semantics, and reveal services with Mode A.

11.8.2 The point-cloud versus 3D-model comparison

The implemented landing experience renders a deterministic split comparison of the two spatial representations Reframe works with (Figure 22): a Gaussian-splat-style point cloud (`fixed-gaussian-room.ts`: a seeded, reproducible room of splats with per-splat radius, rendered through a custom point-sprite shader with Gaussian falloff and depth fade) side by side with a meshed 3D room model (`fixed-room-model.ts`: a fixed part list of boxes for floor, rug, walls, window, sofa, side table, and plant, sharing the same right-handed, $+Y$ -up, metric coordinate convention). Both are presentation assets, never canonical scene or replay data; the module documentation in the repository states this invariant directly.

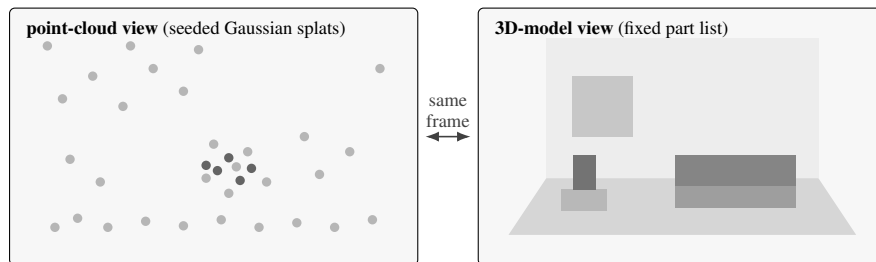


Figure 22: Schematic of the implemented web comparison view: a deterministic seeded splat cloud and a fixed meshed room model rendered side by side in the same right-handed, $+Y$ -up, metric frame. Both are presentation assets; neither is canonical scene or replay data.

11.9 Mode B1 polish worker

Mode B1 consumes an immutable session snapshot; selects 80–160 keyframes by sharpness, coverage gain, baseline, loop value, and semantic relevance; uses only the MapAnything Apache-compatible model and configuration for global metric consolidation; initializes gsplat from consolidated geometry; and optimizes in four phases (fixed poses with SH0; bounded pose and exposure correction with SH1–2; densify/prune; compress/export and map polished primitives back to canonical surfaces). Hot-swapping the polished skin is permitted only when `source_scene_revision` is compatible. B1 can be omitted entirely without reducing core-release completeness.

12 Infrastructure and execution

12.1 Runtime topology and Docker services

The repository layout mirrors the runtime topology: `apps/api` (TypeScript gateway and scene authority), `apps/ios` (native Xcode project, not containerized), `apps/vision` (Python 3.12 live, map, and reveal profiles), `apps/web` (Next.js Mode B client), and `packages/agent`, `packages/catalog`, `packages/protocol`. Table 6 lists the containers and Figure 23 the deployment topology.

Table 6: Docker service inventory.

Container	Runtime	Responsibilities
<code>web</code>	Node	Next.js UI and static assets
<code>api</code>	Bun/TypeScript	auth, sessions, ingest/router, scene revisions, transactions, GPT, tokens, catalog
<code>vision-semantics</code>	Python 3.12	SAM 3.1 stateful target tracking and canonical identity
<code>vision-geometry</code>	Python 3.12	DA3Metric-Large, alignment, TSDF, planes, dense meshes
<code>vision-map</code>	Python 3.12	LingBot-Map ordinary-video reconstruction
<code>vision-reveal</code>	Python 3.12	atlas assembly, deterministic fills, isolated <code>RevealFillProvider</code>
<code>qdrant</code>	Qdrant	persistent semantic vectors, payload filters, catalog references
<code>polish-worker</code>	Python (isolated)	MapAnything + gsplat, disabled by default

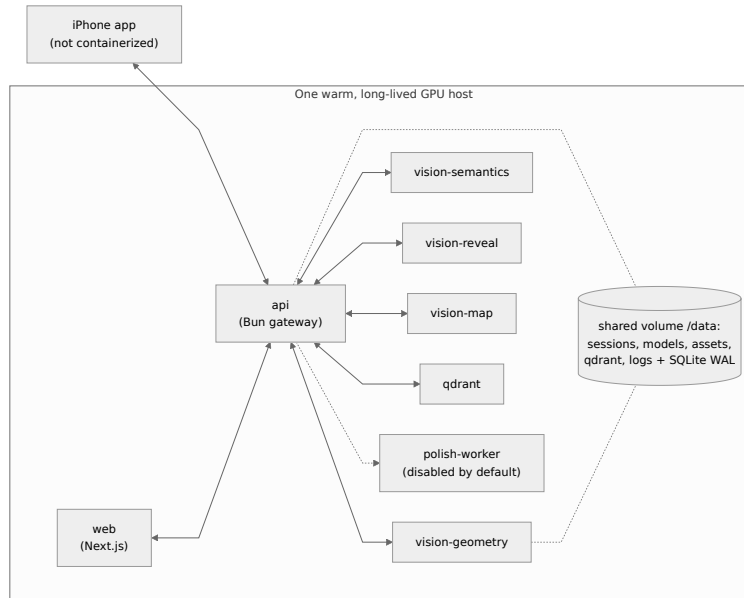


Figure 23: Runtime/Docker topology. The stateful live lane runs on one warm, long-lived GPU host because the active session owns in-memory AR frame state, SAM track state, TSDF blocks, and persistent WebSockets; queue-style serverless jobs are unsuitable for Mode A, although the images remain serverless-compatible for B0/B1 batch work.

12.2 Persistence

One shared host-mounted volume holds `/data/sessions/<session_id>/`, `/data/models/`, `/data/assets/`, `/data/qdrant/`, and `/data/logs/`. The gateway stores session metadata in SQLite WAL. Qdrant is dedicated to catalog retrieval and never becomes scene or transaction authority. Redis, PostgreSQL, S3-compatible storage, and Kubernetes are explicitly excluded absent a measured product need (RF-D12).

12.3 GPU scheduling lanes

One physical GPU is sufficient for the core system if work is prioritized into eight lanes (Figure 24). Only one GPU-heavy job from each lane runs concurrently unless measured otherwise; a second GPU may later isolate semantics and reveal without changing contracts.

The repository implements this as `GpuLaneCoordinator` in the gateway, with exactly the eight work kinds above, bounded pending work, and typed drop reasons (`superseded`, `queue_full`, `mode_a_active`); the recent-frame ring remains retention for semantics and replay, never a compute backlog.

12.4 Transport data planes and authentication

Five distinct data planes separate concerns (Figure 25): control REST (iOS/web to gateway); the binary frame-ingest WebSocket (iOS to gateway ingest/router, which forwards only the latest eligible frame to geometry while retaining the 5 s/64-frame ring for target semantics, so the phone uploads each live frame exactly once); the artifact/event WebSocket (gateway to iOS/web); signed HTTP (`.rfcap`, keyframes, textures, meshes, USDZ/GLB); and the separate iPhone-to-OpenAI GPT Realtime 2.1 WebRTC voice plane.

Authentication: the gateway issues short-lived, room-scoped JWTs with claims for session ID, role, allowed paths, expiry, and nonce; workers reject session mismatches and expired tokens; signed artifact URLs are short-lived. The repository's durable session store implements HMAC-signed room-scoped credentials with a bounded lifetime (at most 15 minutes) and path allow-lists (`frames`, `events`, `artifacts`, `scene`).

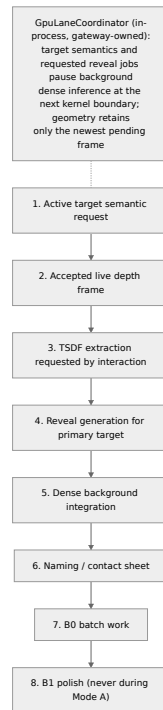


Figure 24: GPU scheduling lanes in strict priority order (1 highest). The coordinator is an in-process gateway component, not an infrastructure service; workers keep models resident but may not launch uncoordinated background GPU work.

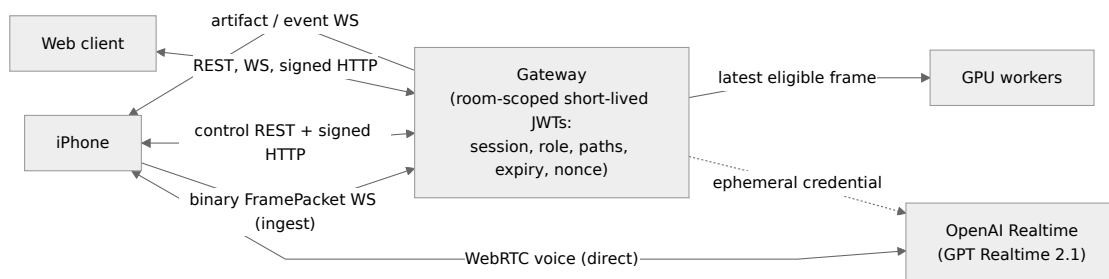


Figure 25: Transport data planes and authentication. Voice bypasses the gateway for media (direct WebRTC to GPT Realtime 2.1) but not for authority: the gateway mints the ephemeral Realtime credential, and the standard API key never reaches the phone.

12.5 Reconnect and idempotency

On reconnect the phone reports its durable position: session ID, last acknowledged frame ID, scene revision, and per-artifact revisions. The server returns a delta when possible, otherwise a full current snapshot (Figure 26). Every event and transaction is idempotent, so replays of the same delta are harmless.

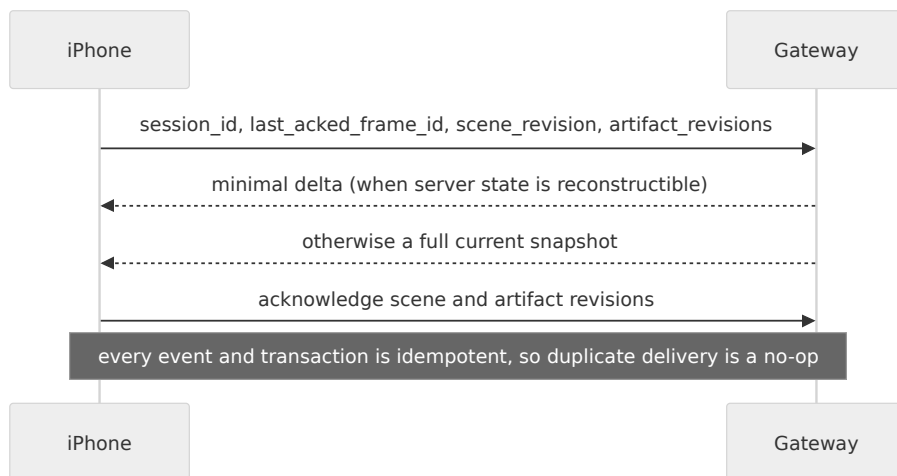


Figure 26: Reconnect sequence. The phone declares its durable revisions; the gateway answers with a minimal delta or a full snapshot, and idempotency keys make duplicate delivery safe.

12.6 Deployment posture

The stateful live lane deploys on one warm, long-lived RunPod Pod or equivalent GPU host. A later load-balancing deployment is acceptable only with one active worker, one maximum worker per session, direct HTTP/WebSocket ingress, and either explicit session affinity or externalized recoverable state. Cold scale-to-zero suits B0/B1 batch jobs, never the primary Mode A session.

13 Performance and resource budgets

All numbers in this section are acceptance targets to be measured, not claims made before profiling. None of them has been validated on a physical device or loaded GPU host at the time of writing (Section 16).

13.1 iPhone budgets

Table 7 and Figure 27 give the on-device targets and their hard fallback thresholds.

Table 7: iPhone performance targets and hard fallback thresholds (targets, not measurements).

Metric	Target	Hard fallback threshold
Camera/compositor render	60 FPS	≥ 45 FPS
Local placement ghost	< 1 rendered frame	< 50 ms
Active-session memory	< 1.2 GB preferred	no jetsam/crash
Thermal state during 4 min	nominal/fair	no sustained serious/critical
Tracking pause response	< 100 ms UI	< 250 ms

13.2 Cloud and interaction latency budgets

Table 8 lists the cloud-side targets; Figure 28 charts the key interaction latencies.

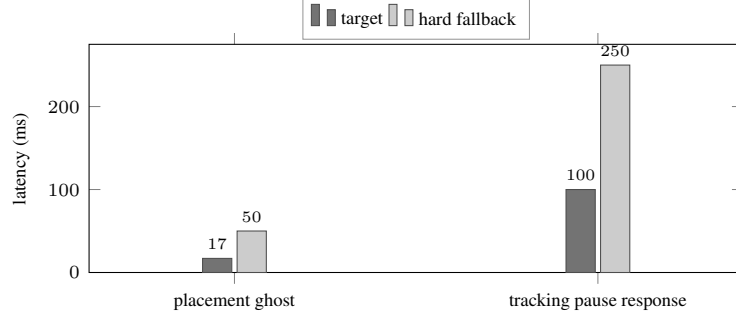


Figure 27: On-device latency targets versus hard fallback thresholds in milliseconds. One rendered frame at 60 FPS is approximately 17 ms; render-rate and memory/thermal budgets appear in Table 7. These are unmeasured acceptance targets.

Table 8: Cloud and interaction targets (targets, not measurements).

Metric	Target
Accepted frame packets	6–12/s
Queue growth over 2 min	zero
Frame → accepted dense-map update p50 / p95	≤ 250 / ≤ 450 ms
Target seed → tracked p50	≤ 0.8 s
Target seed → asset-only or prewarmed replace_ready p50 / p95	≤ 2.0 / 3.5 s
Target seed → progressive composite preview	3–10 s; explicit <i>Healing</i> through 30 s
Warm-up → remove_ready	10–30 s, scene dependent
Simple spoken command → visible change p50 / p95	≤ 2.5 / ≤ 4.0 s
Placement validation excluding LLM	≤ 300 ms
Commit delta → local activation	≤ 250 ms after artifacts cached

13.3 Artifact size budgets

Table 9 and Figure 29 give the core-release artifact budgets.

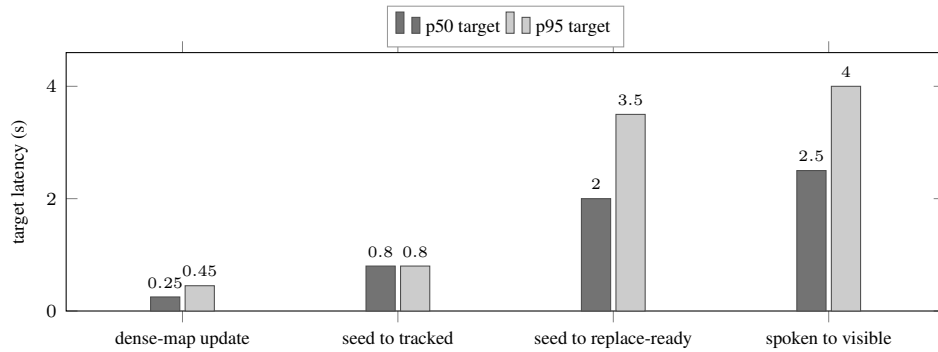


Figure 28: Interaction latency percentile targets in seconds. For the seed-to-tracked metric only a p50 target is specified, so both bars coincide. All values are unmeasured targets.

Table 9: Artifact size budgets (core release).

Artifact	Core-release budget
Mask volume / object	≤ 1 MB compressed
Surface mesh / object	≤ 1.5 MB
Reveal bundle / object	≤ 4 MB preferred
Initial occluder mesh set	≤ 5 MB
USDZ asset	≤ 15 MB; prefer ≤ 10 MB; all primary candidates bundled/pre-cached
Incremental scene delta	≤ 100 KB excluding signed artifact downloads
High-resolution keyframe	≤ 1.5 MB preferred; sparse async upload

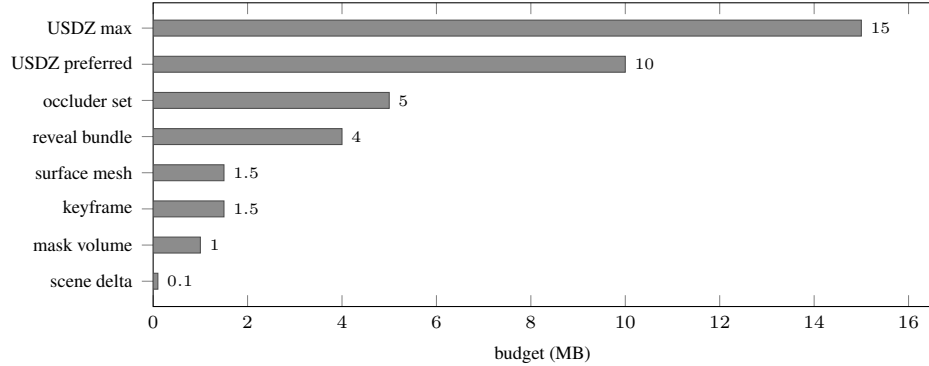


Figure 29: Per-artifact size budgets in megabytes (scene delta shown at its 0.1 MB budget). Budgets bound download-and-activate latency on the phone and keep commit-time caching predictable.

14 Telemetry and evaluation

Per-frame and per-job tracing logs: capture timestamp and frame ID; encode, write, queue, uplink, decode, provider, alignment, fusion, semantic, reveal, download, and activation timings; queue depth and dropped-frame reason; ARKit tracking state; model/checkpoint/container revision; active TSDF blocks and extraction size; phone FPS, memory, and thermal state; scene and artifact revisions; and the transaction tool trace with the deterministic validation result. Raw audio transcripts and image content are never logged into ordinary application logs.

Product quality metrics: command-to-visible-change latency by operation; target-to-track and target-to-capability latency; mask IoU on approximately 20 hand-annotated frames; cover score and reveal-view coverage; synthesized-textel fraction versus observed viewpoints; floor RMSE and taped-distance errors; anchor drift over a three-minute fixed-marker test; provider bake-off results; end-to-end success rate; and phone FPS/thermal stability.

15 Privacy, security, retention, and supply chain

15.1 Privacy and retention

Explicit camera/upload consent precedes every session, with visible recording and network status. All transport is encrypted. The OpenAI standard API key remains server-side; the iPhone receives only short-lived Realtime credentials, and worker tokens are room-scoped and short-lived. Raw uploaded frames are deleted by default within 24 hours after processing unless the user explicitly retains the session for B1. Local `.rfcap` remains under user control and is deletable from the app. There is no training reuse without separate opt-in consent. Consent copy distinguishes self-hosted room-frame processing from OpenAI processing of microphone audio and the small selected object crops and structured scene facts used for naming or planning; full raw room video is not sent to OpenAI by the product architecture. Session deletion removes frames, masks, depth, geometry, reveal textures, logs containing object labels, and database rows. Golden generated development test cases require documented participant consent and are stored separately from user sessions.

15.2 License inventory and supply-chain rules

Before a model or asset enters the core release, the exact repository, commit/checkpoint hash, license, and download source are recorded (Table 10); model preparation is explicit, and every checkpoint is verified before service startup.

The repository already pins concrete provenance for its providers. The DA3 integration records the exact source revision, checkpoint byte length, and SHA-256, and the SAM 3.1 provider names its checkpoint repository and file, consistent with this rule.

Table 10: License and supply-chain inventory rules.

Component	Rule
DA3Metric-Large	Apache-2.0 checkpoint; permitted candidate
DA3-Small/Base	Apache-2.0 checkpoints; permitted candidates
DA3 Large/Giant/Nested variants	do not accidentally adopt non-commercial variants without review
LingBot-Map	Apache-2.0 repository; pin commit/checkpoint
SAM 3.1	SAM License; team reviews and records acceptance before use
Open3D	pin release and license record
LaMa	first provider uses pinned official PyTorch implementation/checkpoint on CUDA; isolate, hash, review exact license; adopt ONNX only after measured parity
MapAnything	Apache-compatible model/config only
gsplat	pin release/commit and license record
Spark	B1-only web dependency; pin release/commit
Assets	every USDZ/GLB pair has source, attribution, and license ledger

16 Implementation state and verification

This section separates (a) what the Master Technical Prompt specifies as design authority, (b) what is implemented and verified in the repository today, and (c) the acceptance gates that remain open. Nothing in Section 13 or elsewhere should be read as a measured result.

16.1 Specified design

Everything in Sections 1–15 is normative specification from the Master Technical Prompt. It is the authority against which the implementation is built and reviewed; where the repository and the prompt disagree, the prompt governs and the code is the defect.

16.2 Implemented and verified in the repository

Table 11 maps subsystems to their concrete implementation and test evidence. In summary:

- **Protocol package.** FramePacket encode/parse with the exact 24-byte header, magic, flags, and metadata schema; the RF-COORD-1 coordinate conventions including $C_{\text{arkit} \leftarrow \text{opencv}}$ and row-major serialization; canonical JSON hashing; capture and coordination events including `target_seed`; the transaction/proposal model with idempotency keys and CAS revisions; placement preview; and `evaluateReplacementCover` implementing the exact thresholds of Section 10.5, including the eight-view minimum and conservative quantiles. All of this is covered by unit tests.
- **Gateway.** A durable SQLite-WAL session store with HMAC-signed, room-scoped, path-scoped, expiring credentials; durable frame receipts and coordination events; a durable edit-transaction service; agent-turn services binding typed turns to durable spatial context (the change that removed client-supplied spatial authority); the five read-only agent tools; and the in-process `GpuLaneCoordinator` with the eight prioritized lanes and typed drop reasons.
- **Agent package.** The bounded tool runner (call and candidate limits), OpenAI Realtime session and WebRTC clients for GPT Realtime 2.1, planner adapters, and model-capability guards, with tests.
- **Catalog package.** Acquisition stores, IKEA source authorization and download transport, GLB validation, Blender-based normalization processors, preparation and delivery, client cache sync, eligibility filtering, Qdrant store and retrieval, OpenAI enrichment and vectorization, and smoke-test commands, with an extensive unit-test suite.
- **Vision workers.** Typed provider boundaries with honest readiness reporting (a `DisabledProvider` refuses work rather than fabricating results); DA3 integration with pinned source revision, checkpoint SHA-256 and byte length, and the repository metric formula; a SAM 3.1 provider with pinned checkpoint identity and per-session track stores; conservative silhouette geometry; reveal quality-gate constants exactly matching Section 11.4.3; and an isolated `LaMa RevealFillProvider` wire contract with hash-verified outputs. Python tests cover wire, providers, tracks, and geometry.

- **iOS SpatialCore.** Swift packages for the FramePacket, target seeds, room sessions, .rfcap capture with atomic persistence, frame selection, replacement-target projection, local placement preview, asset delivery, and a scene replica that retains committed edits, with Swift tests.
- **Web client.** Capture session brokering and upload, replay data handling, delivered assets, Realtime room wiring, and the deterministic point-cloud versus 3D-model comparison (Section 11.8), with unit tests. The browser-video path was explicitly corrected to carry no fabricated ARKit authority.

Table 11: Verification matrix: implementation and test evidence by subsystem (repository state at the time of writing).

Subsystem	Implemented artifacts	Evidence
Protocol contracts	FramePacket, coordinates, canonical JSON, events, transactions, placement preview, replacement cover	unit tests in <code>packages/protocol/test</code>
Gateway	durable session store (SQLite WAL, HMAC room JWTs), durable transactions, agent turn binding, read-only tools, GPU lane coordinator	<code>apps/api/src</code> + <code>apps/api/test</code>
Agent adapters	bounded tool runner, Realtime session/WebRTC clients, planner, capability guards	unit tests in <code>packages/agent/test</code>
Catalog	acquisition, authorization, validation, normalization, preparation, delivery, Qdrant retrieval, cache sync	extensive unit tests + smoke commands in <code>packages/catalog</code>
Vision workers	provider boundaries, DA3 + SAM 3.1 pinned identities, conservative geometry, reveal gates, LaMa wire contract	Python tests in <code>apps/vision/tests</code>
iOS SpatialCore	FramePacket, .rfcap atomic capture, target seeds, projection, placement preview, scene replica	Swift tests in <code>apps/ios/Packages/SpatialCore/Tests</code>
Web Mode B0	capture broker/upload, replay data, delivered assets, deterministic room comparison	unit tests in <code>apps/web/src/lib</code>

16.3 Open gates

Table 12 lists the acceptance gates that are specified but not yet proven. These are not incidental: each requires hardware, a live model deployment, or scale that unit tests cannot substitute for.

Table 12: Open acceptance gates: specified but not yet demonstrated.

Gate	What remains to be proven
Real SAM-driven replacement	an end-to-end replace on a live SAM 3.1 track (not fixtures), through cover-score validation to a committed transaction
LaMa and TSDF in production	the isolated LaMa fill worker and the Open3D CUDA TSDF running against real session data at budgeted cadence
Live Realtime/Qdrant acceptance	voice turns through GPT Realtime 2.1 and retrieval against a populated Qdrant instance under live-session conditions
Physical-device gates	iPhone FPS, thermal, memory, and the half-circle visual review of compositor and reveal quality (Table 7 targets)
IKEA-scale ingestion	the full authorized catalog frontier ingested, normalized, and indexed at scale (beyond smoke-tested components)
Browser video-mapping ingress	ordinary browser or uploaded video through LingBot-Map into a session twin
Latency/queue budgets	every target in Tables 8 and 9 measured under load

16.4 Interpretation

The pattern across Tables 11 and 12 is deliberate. The deterministic core (contracts, coordinate conventions, transactions, readiness math, scheduling, durability, and provider boundaries) is implemented and tested first, because every open gate depends on it and none of it depends on a GPU. The open gates are integration and measurement work, not design work; closing them changes no contract in this report.

17 Conclusion

Reframe shows that a live diminished- and augmented-reality editor can be built on a small number of structural commitments: deterministic spatial authority with advisory models; two parallel geometry tracks that keep learned reconstruction off the interaction-critical path; a capability-specific readiness model that does not promise what the system cannot yet render; conservative geometry and cover-score gates that prefer overcoverage to visible fragments of the real world; typed, versioned, hash-verified contracts at every runtime boundary; and infrastructure sized to the

product’s measured needs. The repository verifies the deterministic core of this design today; the remaining work is to close the acceptance gates of Section 16.3 without weakening any invariant documented here.

Acknowledgements

Two AI systems deserve explicit credit for the pace and quality of this project. GPT-5.6 Sol is not only the runtime planner documented in this report; throughout development it proved an exceptional engineering collaborator, reasoning cleanly about contract boundaries, catching inconsistencies between the specification and the implementation, and turning loosely stated constraints into precise, schema-ready definitions. OpenAI Codex carried a remarkable share of the implementation itself: it scaffolded the cross-runtime protocol adapters, wrote and refactored large parts of the gateway, catalog, and vision workers together with their test suites, and kept the Swift, TypeScript, and Python implementations of the same wire contracts consistent with one another. A system of this scope, built in one intensive week by a four-person team, would not have reached its current implementation depth without both of them. Every contribution from either model was reviewed, tested, and validated against the Master Technical Prompt before it entered the repository.

18 Disclosure of LLM and AI Tool Use

Large language models and AI tools were used at several stages of this project and in preparing this report, as disclosed below.

- **Whether LLMs or other AI tools were used:** Yes. OpenAI Codex and GPT-5.6 Sol were used extensively as development assistants for the Reframe system, and an LLM was used to draft and typeset this report from the project’s authoritative technical documentation and source code. The system under description itself integrates LLMs as runtime components (GPT-5.6 Sol for planning and naming, GPT Realtime 2.1 for voice), as documented in the body of the report. No AI image generators were used; all figures are vector diagrams specified by the authors (Mermaid and TikZ/PGFPlots).
- **For which tasks they were used:** brainstorming and refining the system architecture and its contracts (GPT-5.6 Sol); writing and refactoring implementation code and tests across the Swift, TypeScript, and Python codebases (Codex); and drafting, structuring, and editing the text, figures, and tables of this report.
- **How correctness of LLM-generated content was verified:** every technical claim in this report was checked against the project’s Master Technical Prompt (the product and architecture authority) and against the repository source code and its test suites; all generated code was reviewed by the authors and gated by the unit-test suites described in Section 16; implementation claims were restricted to code and tests actually present in the repository; performance figures are explicitly labeled as unmeasured targets rather than results; and the report was compiled and reviewed by the authors before submission.